What Is a ReLU Turing Machine?

Exact Compilation, Learned Controllers, Writable Tapes, Verification, Text Machines, and Symbolic Search

Working Draft

August 2026

Abstract

This working draft consolidates a sequence of mathematical notes and experiments around a simple architectural idea: keep the tape, head positions, reads, writes, and moves of a Turing-style machine discrete, but implement its finite local transition controller with a feed-forward ReLU network. The term *ReLU Turing Machine* (ReLU-TM) is used here as a working term, not as established nomenclature.

The central theoretical fact is elementary but operationally useful. Every finite deterministic Turing transition table can be compiled exactly into a finite ReLU network with explicitly chosen weights. If the neural controller is correct on every local configuration that a deterministic run can encounter, then the complete execution is identical to the original machine by induction. Because the local domain is finite, a learned controller can also be exhaustively verified. This yields a compiler continuum: exact constructive compilation, exact symbolic compression, learned compression, learned-and-verified compression, and learned compression plus an exact exception table.

The experimental path was less direct. Early text experiments induced finite reader machines from *Alice's Adventures in Wonderland*. A growing Fourier-weighted network learned encountered state transitions well but produced weak next-character predictions and repetitive free runs. A plain ReLU MLP improved the predictive metrics substantially, yet still failed to recover the complete state-by-alphabet table. Exact constructive work then shifted attention from language generation to finite transition programs. Hand-set ReLUs represented thousands of transitions exactly; a hand-set ReaderFWNN represented two unrelated machines exactly; a trained shared ReaderFWNN became very accurate but not exact. In a five-tape modular-multiplication controller with 39 states and 596 defined rules, this distinction became decisive: a learned ReaderFWNN at roughly 97.6–97.8% local accuracy failed long computation, whereas a plain 59–64–64–64 ReLU network learned all 596 rules exactly in 10/10 random seeds. The resulting verified controller was executed on fresh cases with operands up to 4000 bits and nearly 99 million Turing transitions.

Writable-text experiments exposed the same threshold from another angle. A 35,773-parameter ReLU achieved 99.962% joint local accuracy but 0/500 successful repairs because it missed one mandatory transition from left-seeking to copying. Restoring trajectory frequency and then adding counterfactual local coverage produced 100% local test accuracy and 700/700 exact held-out and stress reconstructions, including snippets up to 2048 characters. Follow-up controls showed that this network had learned a navigation-and-copy procedure, not the text of *Alice*. Finally, a small symbolic proof gym demonstrated a related hybrid pattern: greedy neural tactic selection generalized poorly to two unseen proof motifs, but the same ReLU policy inside exact DFS/backtracking proved every held-out theorem; random DFS did so as well, so no neural search-efficiency advantage was established.

Taken together, the results support a narrow but coherent view: neural weights can serve as one implementation target for a finite symbolic controller, while external symbolic memory and exact verification provide the long-horizon semantics. The most important distinctions are therefore *representability*, *learnability*, *compression*, *algorithm discovery*, and *long-run execution*. They are related, but they are not the same claim.

Contents

1 The research path: from finished text to verified neural controllers

The project began from a text-learning question rather than from a compiler theorem. If only a finished text is given, can one induce a finite machine whose local transitions can then be learned by a neural network? The first attempts treated the text as a source of pseudo-Turing traces. That route produced useful failures: state transitions could be predicted extremely well along the observed text distribution while autonomous generation remained poor. The gap between local accuracy and stable execution then motivated a more precise question: *what, exactly, must the neural component represent?*

The answer that emerged was to separate the finite control law from the memory substrate. A classical deterministic Turing machine already supplies a finite local program: given a finite control state and the symbols under the heads, choose a new state, symbols to write, and head moves. The unboundedness of the computation does not come from an unbounded controller. It comes from repeatedly applying a finite controller to an external tape that can grow as needed.

This re-framing led to a sequence of stages summarized in table 1.

The accumulated lesson is not that ReLU networks are a new universal computer in the abstract. Neural simulation of automata and Turing computation is longstanding. The useful point is more engineering-oriented: a finite transition program can be made into a neural artifact, that artifact can be trained or constructed, the complete local domain can sometimes be enumerated, and exact local behavior can be connected rigorously to exact global execution.

2 Definition of a ReLU Turing Machine

2.1 Deterministic k-tape controller

Consider a deterministic k-tape Turing machine with finite state set Q, finite tape alphabet Γ, and partial local transition function

$$\delta : D \subseteq (Q \setminus H) \times \Gamma^k \longrightarrow Q \times \Gamma^k \times \{L, S, R\}^k, \tag{1}$$

where $H \subseteq Q$ is the set of halting states. For a local configuration

$$c = (q, s_1, \ldots, s_k) \in D, \tag{2}$$

write

$$\delta(c) = (q', w_1, \ldots, w_k, m_1, \ldots, m_k). \tag{3}$$

The tape contents and the head positions are not neural variables. They remain in a conventional discrete simulator. At each step the simulator reads the state and the symbols currently under the heads, calls the neural controller, decodes its output into a discrete transition, writes the symbols, moves the heads, and repeats.

Definition 2.1 (ReLU Turing Machine)**.** A *ReLU Turing Machine* in the sense used in this draft is a hard discrete Turing-style machine whose finite local transition controller is implemented by a feed-forward ReLU network F_θ. If $e(c)$ is a finite-dimensional encoding of a local configuration and D_{dec} decodes network outputs into a discrete transition, exact realization means

$$D_{\text{dec}}(F_\theta(e(c))) = \delta(c) \qquad \text{for every defined local configuration } c. \tag{4}$$

Separation of roles. The network implements the finite local control law. The tape implements persistent working memory. A small controller can therefore participate in a computation that manipulates thousands of bits or runs for tens of millions of steps without storing those bits in its hidden activations.

2.2 Architecture diagram

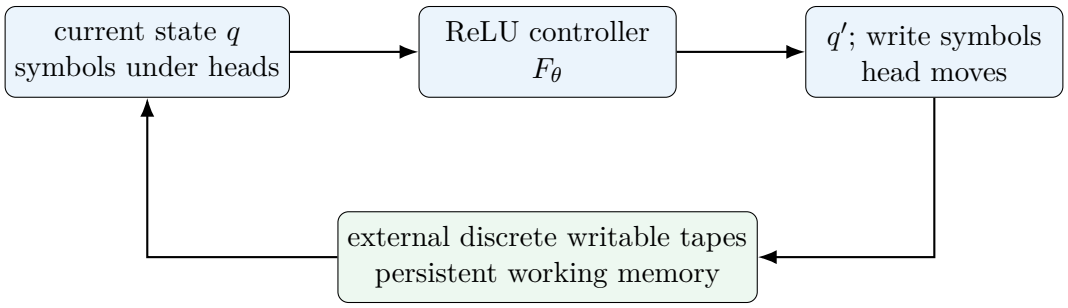

Figure 1: A ReLU-TM keeps the memory mechanics discrete and uses the neural network only as the local controller.

3 Exact constructive results

3.1 Every finite deterministic transition table has an exact ReLU realization

The first theorem is a finite lookup construction, but its importance is conceptual: it cleanly separates *representability* from *learning*.

Theorem 3.1 (Exact ReLU compilation of a finite k-tape transition table)**.** *For every finite deterministic k-tape transition table δ there exists a finite feed-forward ReLU network whose weights can be written explicitly so that the decoded network output agrees with δ on every defined local configuration.*

Proof. Encode the current state and each read symbol in separate one-hot blocks:

$$x(q, s_1, \dots, s_k) = e_q \oplus e_{s_1} \oplus \cdots \oplus e_{s_k}. \tag{5}$$

For every defined transition row

$$p = (q_p, s_{p,1}, \dots, s_{p,k}) \in D, \tag{6}$$

create one hidden unit

$$h_p(x) = \mathrm{ReLU}\left(x_{q_p} + \sum_{i=1}^{k} x_{i,s_{p,i}} - \left(k + \frac{1}{2}\right) \right). \tag{7}$$

On a legal one-hot input equal to row p, all $k+1$ required features are one, hence

$$h_p = (k+1) - (k + 1/2) = 1/2. \tag{8}$$

For any other legal row, at least one required feature is absent, so the sum is at most k and the ReLU output is zero. Therefore exactly one hidden unit fires, with value $1/2$, on each defined input row.

Now create categorical output heads for the next state, each tape's write symbol, and each head move. If

$$\delta(p) = (q'_p, w_{p,1}, \ldots, w_{p,k}, m_{p,1}, \ldots, m_{p,k}), \tag{9}$$

place output weight 2 from h_p to every correct target logit and zero to the alternatives. The correct logits become 1, the incorrect logits remain 0, and argmax decoding recovers the full transition exactly. All weights are obtained mechanically from the table; no optimizer is required. □

Remark 3.2. The theorem is not an efficiency theorem. The direct witness uses one hidden detector per defined transition row. An arbitrary table with little reusable structure may require a representation comparable in size to the table itself. The construction is best viewed as a correctness baseline and a compilation target from which exact compression may begin.

3.2 Exact local behavior implies exact global execution

Theorem 3.3 (Local exactness implies whole-run exactness)**.** *Suppose a neural controller agrees with a deterministic Turing machine's transition function on every local configuration encountered by a run. If the neural-controller simulator and the original machine start from the same complete configuration, then after every finite number of steps they have identical states, tapes, and head positions. If the original machine halts after T steps, the neural simulation halts after exactly T steps with the same tapes.*

Proof. At step 0 the configurations agree by assumption. Assume they agree after step t. Then the state is the same, every tape content is the same, and every head position is the same, so both systems read the same local tuple $(q, s_1, \ldots, s_k)$. By hypothesis the neural controller emits exactly the transition δ emits: the same next state, the same writes, and the same head moves. Applying identical discrete actions to identical configurations produces identical configurations after step $t+1$. Induction proves equality for every finite t. The halting statement follows immediately. □

Why this matters experimentally. In an ordinary classifier, 99.9% may be excellent. In a deterministic controller that must execute millions of sequential decisions, one wrong mandatory transition can destroy the entire computation. Conversely, if the complete reachable finite local table is verified exact, length generalization follows from repeated execution of already verified rules.

3.3 Finitely many unrelated machines in one network

Theorem 3.4 (Finite multi-program compilation)**.** *Let $M_1, \ldots, M_r$ be any finite collection of deterministic Turing machines with finite control and finite alphabets. There exists a single finite deterministic machine U, and therefore by theorem 3.1 a single sufficiently large ReLU controller, that can execute any M_i given a finite selector.*

Proof. Rename each state space disjointly as

$$Q'_i = \{i\} \times Q_i. \tag{10}$$

Construct a dispatcher state that reads or receives a finite selector i and enters $(i, q_{0,i})$. Copy every transition of M_i into the renamed namespace Q'_i. Because the family is finite and each component machine has finite local control, the union is again a finite deterministic transition system. Apply theorem 3.1 to that union. □

This theorem is a representability result. It does not say that gradient descent will discover a compact shared representation or that unrelated programs will compress well when merged.

3.4 Constructive existence for the ReaderFWNN family

The early Alice experiments used a Fourier-weighted architecture called `ReaderFWNN`. The same finite-table idea can be realized constructively in that family if support count, Fourier rank, and latent dimension are allowed to grow.

Theorem 3.5 (Constructive ReaderFWNN realization of a finite one-tape table)**.** *For every finite deterministic one-tape transition table, a sufficiently large ReaderFWNN of the experimental family can be assigned explicit parameters so that it reproduces the table exactly on every defined state-symbol pair.*

Proof. Let the finite states be numbered $q \in \{0, \dots, Q-1\}$ and the alphabet symbols $a \in \{0, \dots, A-1\}$. Disable hidden-to-hidden feedback, set gates and gauges to one, and choose one-dimensional state and symbol embeddings. Use the injective scalar code

$$t(q, a) = q(A + 1) + a. \tag{11}$$

For the N defined transition pairs, sort the codes

$$t_1 < t_2 < \cdots < t_N. \tag{12}$$

Choose latent dimension N and require the latent function to satisfy

$$f(t_i) = e_i \in \mathbb{R}^N, \tag{13}$$

where e_i is the ith standard basis vector.

Any values prescribed on finitely many ordered scalar inputs can be interpolated by a vector-valued piecewise-linear spline. Let

$$s_i = \frac{e_{i+1} - e_i}{t_{i+1} - t_i}, \qquad i = 1, \dots, N-1, \tag{14}$$

and choose $\theta_0 < t_1$. One exact representation is

$$f(t) = e_1 - s_1(t_1 - \theta_0) + s_1 \,\mathrm{ReLU}(t - \theta_0) \tag{15}$$

$$+ \sum_{i=1}^{N-2} (s_{i+1} - s_i) \,\mathrm{ReLU}(t - t_{i+1}). \tag{16}$$

Direct substitution shows $f(t_i) = e_i$ for every table row.

It remains to realize the desired ReLU thresholds and vector coefficients using the architecture's Fourier-generated support parameters. Choose $m = N - 1$ distinct support coordinates $c_1, \dots, c_m$ and use a real Fourier feature bank

$$1, \cos(2\pi c), \sin(2\pi c), \dots, \cos(2\pi R c), \sin(2\pi R c). \tag{17}$$

When $2R + 1 \geq m$, the evaluation matrix can be chosen with full row rank on suitable distinct support points. Therefore the linear systems that map Fourier coefficients to the desired support biases and decoder coefficients have solutions. This sets the thresholds $-\theta_j$ and the spline coefficients exactly. Finally, linear output heads map the basis vector e_i to the correct next state and discrete action for row i. □

The construction is intentionally large: its purpose is to prove that the experimental architecture contains exact finite-table realizations. The later empirical results show that much smaller plain ReLU networks can sometimes learn the same table.

3.5 Exhaustive verification and exact patching

The finite local domain makes a verification strategy available that is usually impossible for large continuous-input neural systems.

Proposition 3.6 (Exhaustive verification certificate)**.** *If every defined row $c \in \mathrm{Dom}(\delta)$ is enumerated and the deployed neural decoder satisfies*

$$D_{\mathrm{dec}}(F_\theta(e(c))) = \delta(c) \quad \text{for all } c \in \mathrm{Dom}(\delta), \tag{18}$$

then the neural controller is exact on the specified table. If every local configuration reachable from a chosen start family lies in that verified table, theorem 3.3 applies to every such run.

Proposition 3.7 (Learn plus exceptions)**.** *Let $E \subseteq \mathrm{Dom}(\delta)$ be the finite set of rows misclassified by a learned controller. A combined controller that uses the exact symbolic rule $\delta(c)$ for $c \in E$ and the neural controller otherwise is exact on all of* $\mathrm{Dom}(\delta)$.

Proof. Every input is either in E, where the exact rule is used, or outside E, where by definition the neural controller already agrees with δ. □

This gives a practical spectrum shown in table 2.

4 Finite texts as Turing-style machines

4.1 n-gram context machines

Let a finite text be

$$T = t_1 t_2 \cdots t_L \tag{19}$$

over a finite character alphabet Σ. Introduce a left boundary symbol $\vdash \notin \Sigma$. For a fixed context order n, define the context before position i as

$$c_i^{(n)} = \mathrm{suffix}_n(\vdash^n t_1 t_2 \cdots t_{i-1}) \,. \tag{20}$$

From the text, define empirical successor counts $N_T(c, a)$ and the conditional distribution

$$p_T(a \mid c) = \frac{N_T(c, a)}{\sum_b N_T(c, b)}. \tag{21}$$

A deterministic mode machine chooses

$$m_T(c) = \arg\max_a p_T(a \mid c), \tag{22}$$

writes $m_T(c)$, moves right, and updates the state to

$$c' = \mathrm{suffix}_n(c \cdot m_T(c)). \tag{23}$$

This is a finite state machine and can be treated as the finite controller of a simple text-writing TM.

4.2 Every finite text becomes deterministic at sufficiently large context

Theorem 4.1 (Finite-text determinization)**.** *For every finite text T of length L, there exists a finite context length n such that every context encountered during sequential reading has a unique observed next character. In particular, $n = L$ suffices when the left side is padded with $\vdash$.*

Proof. Take $n = L$. Before position i, the L-symbol context consists of exactly $L - (i - 1)$ leading boundary symbols followed by the complete prefix $t_1 \cdots t_{i-1}$. Two different positions $i \neq j$ have different numbers of leading boundary symbols and therefore cannot have the same context. Hence every encountered context corresponds to exactly one position of the finite text and has exactly one observed successor. After the last character, an end symbol EOS may be written analogously. □

Corollary 4.2 (Exact finite-text memorization by a ReLU controller)**.** *For every finite text T, there exists a finite text machine and a finite ReLU network that reproduces its local transitions exactly. Starting from the corresponding initial context, the machine can reproduce T character by character and halt at* EOS*.*

Proof. Apply theorem 4.1 to obtain a finite deterministic text transition table and then apply theorem 3.1. Exact execution follows from theorem 3.3. □

This is a capacity statement, not an efficiency or semantics statement. Required capacity can grow with the number of distinct contexts or transition rows.

4.3 Several texts in one network

Theorem 4.3 (Exact multi-text representation with a document selector)**.** *Any finite collection of finite text machines can be represented exactly by one sufficiently large ReLU controller if a finite document identifier is included in the state or input.*

Proof. Use the dispatcher construction of theorem 3.4, for example by renaming each context state as (document ID, context). □

A more informative question removes the document identifier.

Theorem 4.4 (Successor-consistency condition without a document selector)**.** *A deterministic shared next-character controller can represent several finite text transition tables exactly without a document identifier if and only if every context that appears in more than one text requires the same next character in all occurrences across those texts.*

Proof. Necessity is immediate: one deterministic input cannot map to two different outputs. For sufficiency, if all shared contexts are successor-consistent, the union of the transition tables is itself a well-defined deterministic finite function. Apply theorem 3.1. □

This theorem makes the context order n a controlled dial. Small n creates many shared states and therefore pressure toward common local statistics. Large n makes the contexts document-specific and moves the system toward memorization.

4.4 Goethe/Hesse experiment: structure-to-memory continuum

Two separate poems were used as a small controlled experiment: Hermann Hesse's *Im Nebel* (432 normalized characters) and Goethe's *Ein Gleiches* (140 normalized characters). The texts were not concatenated. Separate context machines were built for each order n, and a shared ReLU was also trained across both transition sets.

Continuation experiments supplied a genuine n-character seed and then let the deterministic mode machine continue. Hesse used an 80-character horizon and Goethe a 35-character horizon.

With a two-dimensional document selector, the smallest tested hidden width 16 reached 100% of the complete shared transition table at every tested order; at $n = 32$ the network had 17,760 trainable

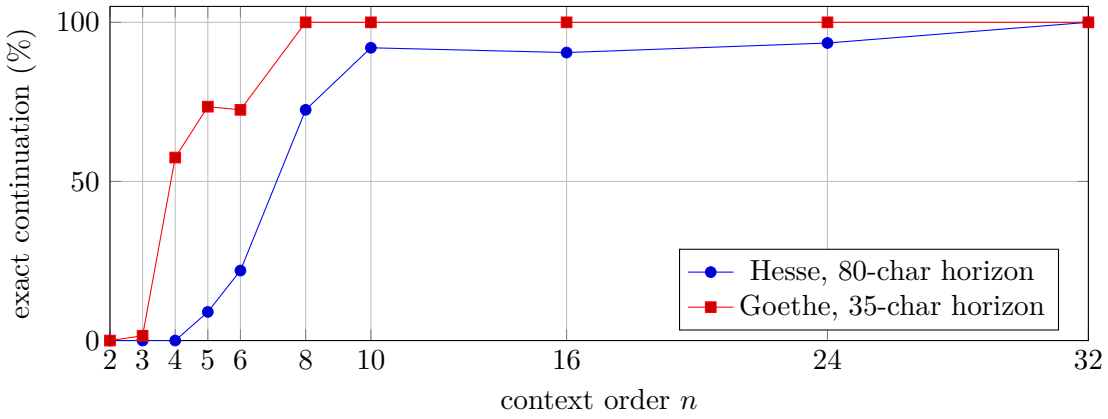

Figure 2: The finite context machine moves from statistical continuation toward near-position-specific memorization as n increases.

parameters. Without a selector, a width-64 ReLU hit the mathematical conflict ceiling exactly at every tested n: for example, 85.854% at $n = 2$ (29 conflicts in 205 table rows) and 99.825% at $n = 32$ (one conflict in 572 rows).

The experiment therefore supports a precise but limited interpretation. Long contexts can force exact memorization of a finite document; this is not evidence of semantic language understanding. Short contexts create genuine parameter-sharing pressure across texts, but evidence for learned shared language structure would require stronger tests on new passages and new documents under a fixed capacity budget.

5 Experimental path I: predictive readers induced from Alice

5.1 The first word-level attempt

The earliest pilot tokenized *Alice's Adventures in Wonderland* at the word level. It used 26,776 tokens, an 800-word vocabulary, 128 latent states, and suffix memory of order at most three. The final 32-support model reached 98.39% test state accuracy but only 17.56% word top-1 and 35.82% top-5, with perplexity 105.79. The bigram top-1 baseline was 23.20%, so the word predictor underperformed a simple bigram despite excellent state prediction. This failure encouraged a shift to character-level modeling where the finite alphabet and transition structure were easier to audit.

5.2 Character-level growing FWNN

The character pilot normalized the book to 143,127 characters over a 49-symbol alphabet and split it chronologically into 80% train, 10% validation, and 10% test. The training prefix induced 128 suffix-reader states, each representing the longest selected frequent suffix up to eight characters. Scanning the text produced local examples of the form

$$(q_t, c_t) \mapsto (q_{t+1}, c_{t+1}). \tag{24}$$

The state transition is deterministic for the induced reader; the next-character target is statistical because the same finite local state can occur at different text positions with different successors.

The growing ReaderFWNN used active support sizes

$$4 \to 8 \to 12 \to 16 \to 24. \tag{25}$$

Candidate supports came from an unbounded Van der Corput stream and were ranked by zero-gate sensitivity. New supports were inserted with zero gates, making each growth event function-preserving up to small floating-point noise.

The held-out final 10% produced:

- teacher-forced next-state accuracy: 99.40%;
- hard predicted-state step accuracy: 99.36%;
- next-character top-1: 31.91%;
- next-character top-5: 66.55%;
- character perplexity: 11.42;
- unigram top-1 baseline: 18.21%;
- bigram top-1 baseline: 30.69%;
- complete 128×49 state-by-alphabet next-state table accuracy: only 69.39%.

The free-running generator collapsed into a short repeated phrase.

5.3 Plain ReLU replacement

To separate the reader representation from the Fourier-weighted architecture, the same induced state problem was trained with a conventional MLP. The input was a 177-dimensional concatenation of a one-hot 128-state vector and a one-hot 49-character vector. Three ReLU hidden layers fed separate next-state and next-character heads.

Five independent width-32 runs at 2500 updates gave mean next-state accuracy 99.926%, mean next-character top-1 43.81%, top-5 76.83%, and perplexity 7.19. Yet mean complete-table next-state accuracy was only 77.53% with a range of 74.57–80.79%. Free-running generation still entered repetitive attractors.

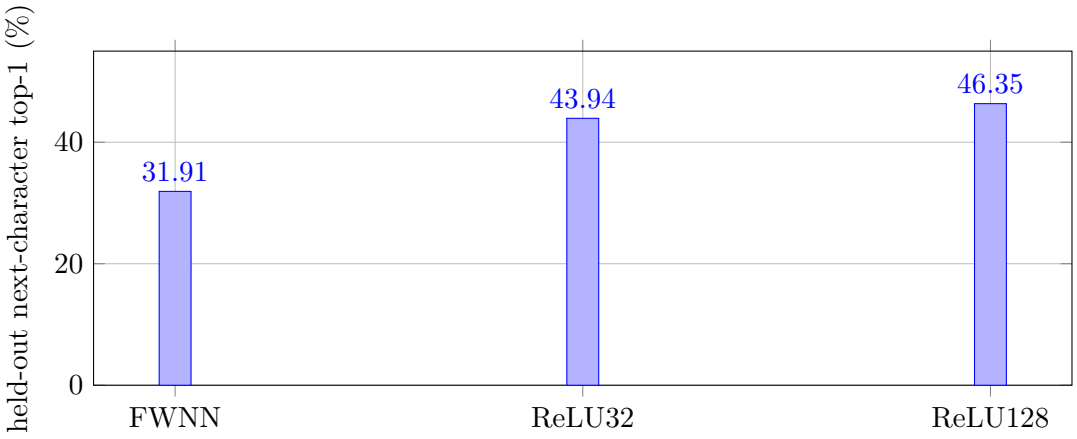

Figure 3: A plain ReLU MLP substantially improved predictive performance, but did not solve the complete-table or free-running problems.

The central negative result from the Alice predictor was therefore distributional: near-perfect teacher-forced state accuracy does not imply recovery of the full transition law, and better local next-character likelihood does not imply stable autonomous text generation.

6 Experimental path II: exact compilation and arithmetic fixtures

The next stage removed language ambiguity and tested deterministic transition tables directly.

6.1 Finite one-tape arithmetic fixtures

A package of JSON Turing machines encoded finite-domain arithmetic test fixtures: addition in bases 2 through 10, binary addition, subtraction, multiplication, division, and a binary dispatcher. The arithmetic factory intentionally compiled finite string functions into genuine deterministic, halting one-tape machines through a trie-like input recognizer followed by output writing. These fixtures were not claimed to implement unbounded arithmetic for arbitrary input lengths; their purpose was to test the transition-table theorem empirically.

Across 14 primary machines there were:

- 116 arithmetic cases, all correct;
- 873 states in the disjoint union;
- 21 scanned symbols;
- 24 distinct combined actions $(\text{write}, \text{move})$;
- 1,781 defined local transition rows.

A separately composed binary dispatcher also passed all 32 of its cases.

6.2 Hand-set exact ReLU

The direct construction of theorem 3.1 was applied to all 1,781 rows of the disjoint union. It achieved 100.0% next-state accuracy and 100.0% action accuracy on every defined transition. This was an empirical check of the explicit weight formula, not a learning result.

6.3 Hand-set exact ReaderFWNN

The constructive ReaderFWNN theorem was then instantiated on the complete disjoint transition tables of two unrelated machines, base-2 addition and binary multiplication. The combined table contained 112 disjoint states and 156 defined transitions. The construction used 155 support nodes, Fourier order $R = 77$, and latent dimension 156. It achieved 100.0% next-state and 100.0% action accuracy without a single gradient update.

6.4 One trained ReaderFWNN across 14 machines

The same 14 arithmetic machines generated 1,886 trace steps and 1,345 distinct actually visited state-symbol pairs. One trained ReaderFWNN, grown to 64 supports, achieved:

- next-state accuracy: 99.405%;
- action top-1: 99.926%;
- action top-5: 100.0%.

This was very close to the finite table, but not exact. That gap became the decisive issue in the next experiment.

7 Writable external memory and modular multiplication

7.1 Why the tape changes the capacity question

A conventional neural arithmetic benchmark often gives the full operands to a network and asks for the complete answer. The experiments here instead use a small local controller and leave the long-lived working data on writable tapes. The neural network therefore does not need to represent a 4000-bit number in its hidden vector. It needs to choose the correct local rule from a finite controller table while the tape stores the numbers.

7.2 First modular pipeline: four learned primitive controllers

An initial SAIR-style modular-multiplication pipeline used four deterministic multi-tape machines for binary add, subtract, compare, and shift-left. A shared ReaderFWNN saw disjoint global states and read-symbol tuples. Short training cases used 2, 3, 4, 5, 6, and 8-bit operands; validation used 10 and 12 bits; held-out tests used 16, 24, and 40 bits.

The learned shared controller had 15 global states, 15 read-tuple tokens, 24 action tokens, and 12 final support nodes. It achieved 100% next-state and action accuracy on the unique train, validation, and test local pairs. Every local pair used in the long tests had already appeared in short training traces (32/32 coverage), so this was a clean *length-generalization* result rather than generalization to unseen local rules.

End-to-end results were 12/12 correct at 16 bits with a mean of 1,489.8 exact TM steps, 10/10 at 24 bits with 3,509.1 steps, and 8/8 at 40 bits with 7,975.1 steps. Python still orchestrated the outer loop at this stage, which motivated construction of one complete full controller.

7.3 The complete five-tape controller

The full modular multiplier is a deterministic five-tape machine with 39 controller states and 596 defined local transitions. It implements binary streaming reduction and a double-and-add multiplication procedure. The long values remain on tapes; the finite controller selects comparisons, shifts, increments/additions, and subtractions.

The accompanying SAIR note deliberately treated this as a research comparison rather than a competition-compliance claim. The controller is weight-dependent and the tape interpreter is generic, but the arithmetic program and the supervised transition labels were manually designed. That provenance sits close to the boundary between learned model and hard-coded circuit, so no official compliance conclusion is asserted here.

7.4 Monolithic ReaderFWNN: representable but not learned exactly

A short curriculum of 280 random prime-modulus cases with 2–12 bit operands visited 534 unique local rows. A learned monolithic ReaderFWNN with 40 support nodes reached approximately 97.57% next-state accuracy and 97.75% action accuracy, with 518/534 observed rows decoded exactly. A first long rollout failed even though the needed local rows had appeared in the short curriculum.

This is not a contradiction of the exactness theorem. The representation theorem says exact weights exist; it does not say that a small architecture and gradient descent must find them.

7.5 Constructive exact ReaderFWNN and large runs

The complete 596-row table was then compiled constructively into a large ReaderFWNN with 595 support nodes and Fourier order $R = 297$. It achieved 100% next-state and 100% action accuracy on all 596 rows without gradient training. The finite predicted table was cached once and executed by a compiled simulator, which still performed every discrete Turing write and head move.

Selected fresh runs included:

These runs are evidence for the induction story at large execution lengths, not evidence that the oversized constructive ReaderFWNN is an efficient neural representation.

7.6 Plain ReLU: exact learning becomes easy

The most decisive baseline removed Fourier structure, growth, attention, normalization, and residual connections. The network was simply

$$59 \to 64 \to 64 \to 64 \tag{26}$$

with ReLU activations and eleven categorical output heads: one next-state head, five write-symbol heads, and five head-move heads. It contained 16,970 trainable parameters.

Training used all 596 defined rules uniformly. The success criterion was not a high average score; training stopped only when all 596 rows decoded exactly. Across ten independent random seeds, all ten runs reached 596/596 exact transitions in 375–550 optimizer steps.

After training, the deployed network was enumerated once on all 596 defined rows. The decoded table had zero mismatches against the target JSON. Large runs then used those deterministic neural decisions as a cache.

The 4000-bit result should be interpreted carefully. The MLP does not contain a 4000-bit multiplication table in its hidden representation. It contains a 596-row local controller. The long integers live on the tapes, and the same verified finite rules are reused almost 99 million times.

7.7 Why 100% is qualitatively different from 99%

A crude independent-error calculation illustrates the scale: if per-step correctness were p, survival for T steps would be p^T. The assumptions are unrealistic for deterministic controller errors, but the magnitude is instructive. For T in the millions, $p = 0.99$ is effectively useless. The finite-controller setting offers a stronger alternative: enumerate the local table and remove the uncertainty entirely on the verified domain.

The modular experiments therefore separated three cases:

- learned ReaderFWNN near 98%: long run fails;
- hand-set ReaderFWNN at 100%: long run succeeds;
- trained plain ReLU at 596/596: long run succeeds.

8 Writable Alice: bidirectional navigation, repair, and a critical failure

8.1 Why the representation had to change

The first Alice reader only moved right. It could predict the next symbol but had no reason to seek left, revisit earlier tape locations, or overwrite persistent memory. A stronger benchmark placed a clean text snippet on a read-only source tape and a corrupted equal-length copy on a writable work tape. Both heads started independently at random interior positions. The controller had to seek left until both boundary markers were found, then move right in lockstep, rewriting the work tape from the source.

The controller had 130 states: one `SEEK` state, 128 `COPY-q` states carrying the earlier suffix-reader state, and one halt state. The alphabet contained 53 tape symbols: 49 normalized text characters plus left marker, right marker, mask, and blank. The ReLU input was

$$130 + 53 + 53 = 236 \tag{27}$$

one-hot dimensions, followed by three hidden layers of width 64. Four output heads predicted next state, work-tape write symbol, source-head move, and work-head move. Total trainable parameters: 35,773.

8.2 Correctness of the symbolic repair machine

Proposition 8.1 (Exact symbolic repair from arbitrary interior head starts)**.** *For every finite equal-length source/work pair with correct boundary markers, and for every pair of initial interior head positions, the symbolic repair machine halts with the work-tape interior exactly equal to the source-tape interior.*

Proof. During `SEEK`, let $d_1, d_2 \geq 0$ be the distances from the source and work heads to their left markers. On each nonterminal seek step, every positive d_i decreases by one while a zero distance remains zero. Therefore after at most $\max(d_1, d_2)$ seek steps both heads are simultaneously on the left markers. The next transition moves both heads to the first content cells and enters `COPY`.

Now induct on copied positions. Before the first copy step both heads point to position 1. Assume after k copy steps that work positions $1, \ldots, k$ equal the source positions and both heads point to position $k + 1$. The controller reads the source symbol x_{k+1}, writes exactly x_{k+1} to the work tape, and moves both heads right. The invariant therefore holds after $k + 1$ steps. After all L characters are copied, both heads reach the right markers and the machine halts. The proof is independent of the text content and of L. □

8.3 Training data and counterfactual coverage

The final run used 6,000 random training snippets of length 24–96, 800 validation demonstrations, 800 held-out local-test demonstrations, 612,608 raw training TM steps, and 45,645 distinct local inputs observed in training trajectories. Only 86.94% of the unique held-out local-test inputs appeared literally in training trajectories.

The final version also generated 59,769 counterfactual local coverage examples from the symbolic oracle. These covered the small SEEK domain and, for every retained reader state and source character, varied the corrupted work symbol. This explicitly taught the algorithmic invariance that the write target during copying is the source character regardless of the current damaged character.

8.4 The key failure: 99.962% local accuracy, 0/500 programs

The first optimized version deduplicated roughly 600,000 demonstration steps into unique local rules and sampled them uniformly. That seemingly reasonable choice dramatically downweighted the one transition

$$(\texttt{SEEK}, \langle L\rangle, \langle L\rangle) \to (\texttt{COPY-root}, R, R), \tag{28}$$

which occurs once in every successful program run but becomes only one row among roughly 45,000 after deduplication.

The model achieved 99.9622% joint local test accuracy and nevertheless reconstructed 0/500 snippets and failed to halt. It moved left correctly but never crossed the mandatory control bottleneck into copying.

The next version restored trajectory-frequency information while retaining some uniform coverage. Its abstract joint local accuracy was slightly lower, 99.9055%, but end-to-end performance jumped to 96.2% exact standard reconstruction and 91.5% exact stress reconstruction, with roughly 98.2–98.5% halting.

The final coverage-augmented version reached 100% on every local output head over 15,872 distinct held-out local inputs. It then achieved:

- 500/500 exact standard reconstructions, lengths 24–512, all halted correctly;
- 200/200 exact stress reconstructions, lengths up to 2048, all halted correctly;
- 0/300 exact reconstructions for an always-right baseline.

The standard cases averaged 274.35 TM steps, 163.08 left-head moves, 328.88 right-head moves across the two tapes, and 82.05 changed writes. The stress cases averaged 1,612.66 TM steps, 1,002.51 left-head moves, 1,933.52 right-head moves, and 531.44 changed writes. Along the model's own stress trajectories, mean next-state agreement with the symbolic oracle was 99.9995%, while every final tape was nevertheless exact; the residual reader-state discrepancy did not occur on a path where it changed the copy/write behavior before halting.

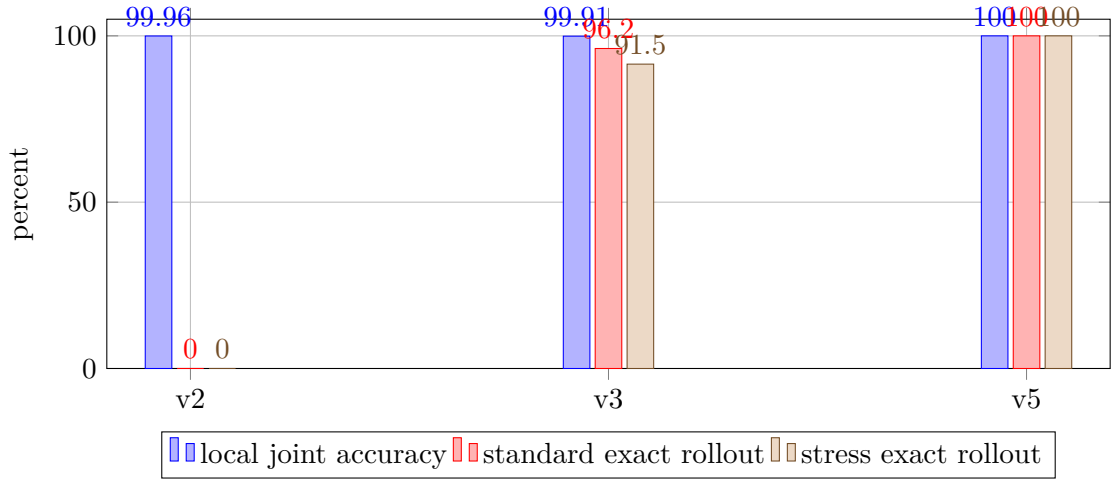

Figure 4: Aggregate local accuracy can move in the opposite direction from whole-program reliability. Which transition is wrong matters more than the average count of wrong transitions.

8.5 Copying versus memorizing

A follow-up addendum tested whether the controller had actually stored *Alice* in its parameters or had learned a content-independent navigation-and-copy procedure.

Five explicitly non-Alice strings were copied exactly. More strongly, 2,000 random out-of-distribution source strings of length 8–512, using the same printable tape alphabet and the same corruption process as training, were reconstructed 2000/2000 exactly by the FP32 controller. When the corruption process was made more adversarial by allowing arbitrary work-tape symbols never seen in training, exactness fell to 63.5%.

The strongest control replaced the clean Alice source with 60 copies of the letter `x`, while keeping a damaged Alice-like target on the work tape. The controller produced 60 `x` characters. It did not recover Alice. The reconstruction was source-driven.

A rough compression diagnostic compared gzip sizes:

The FP32 weights were actually less gzip-compressible than the text because of numerical noise. The simple INT8 representation was smaller than the normalized text under gzip and still copied the tested OOD strings, which is consistent with a function much simpler than the book itself. It is not evidence that the network contains a compressed copy of the book.

9 Neural policy plus exact symbolic search: a mini proof gym

The same architectural separation can be applied outside literal tape machines. A small proof experiment used a symbolic kernel for an intuitionistic propositional fragment over atoms A through E, with implication $\rightarrow$, conjunction $\wedge$, and disjunction $\vee$. The kernel alone determines tactic legality. The neural network only scores which tactic to try next.

9.1 Kernel, tactics, and the first representational failure

The initial tactic language contained `intro`, `constructor`, `left`, `right`, `assumption`, and indexed `apply`, `cases_and`, and `cases_or`. Version 0 exposed a genuine representational limitation: the theorem family

$$(A \rightarrow B \wedge C) \rightarrow (A \rightarrow B) \wedge (A \rightarrow C) \tag{29}$$

could not be expressed because the language lacked a way to materialize the intermediate result $B \wedge C$ from a hypothesis $A \rightarrow (B \wedge C)$ and a proof of A. Version 1 added indexed `specialize` actions. After this change, all generated theorem instances had machine-checked oracle proofs.

BFS generated shortest proof traces for supervision. A plain ReLU policy mapped a 204-dimensional symbolic proof-state feature vector through

$$204 \rightarrow 128 \rightarrow 128 \rightarrow 64 \rightarrow 37 \tag{30}$$

tactic scores.

Training contained 332 theorems and 1,440 state-action pairs. Evaluation used 84 held-out instances from seen theorem families and 78 theorems from three entirely held-out families: associativity of conjunction in one direction, distribution of conjunction over disjunction, and common-target disjunction elimination.

9.2 Results

Teacher next-action accuracy was 97.08% on training states, 97.37% on held-out instances from seen families, and 74.07% on wholly held-out families.

End-to-end greedy decoding performed much worse on the new proof structures:

- seen-family holdout: 74/84;

- wholly held-out families: 26/78;
- breakdown on held-out families: 26/26 for conjunction associativity, 0/26 for distribution, 0/26 for common-target disjunction elimination.

Using the *same* neural scores to order DFS/backtracking repaired the mistakes:

- seen-family holdout: 84/84;
- wholly held-out families: 78/78.

Unguided BFS and random-order DFS also solved 78/78 held-out-family theorems. Median expanded nodes on those held-out families were 17 for policy-guided DFS, 13 for random DFS, and 30 for BFS.

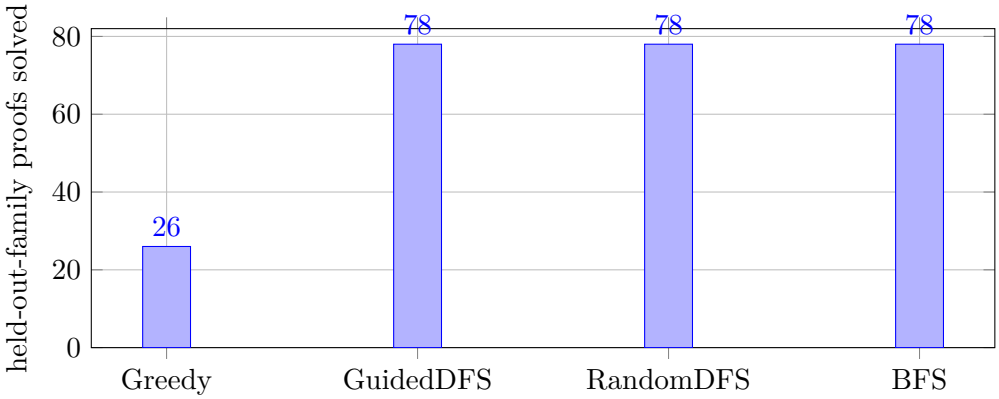

Figure 5: Neural greedy decoding did not generalize compositionally to two unseen proof motifs. Exact symbolic search repaired the policy, but the tiny benchmark did not show a search-efficiency advantage over random DFS.

This experiment supports the broader hybrid principle but not a strong performance claim: a learned heuristic can guide an exact symbolic search process, while the symbolic kernel preserves validity. On this benchmark the search space is still too easy to establish that the neural ordering is better than an unguided heuristic.

10 The compiler view

The accumulated results are most naturally organized as a compiler rather than as a single training recipe.

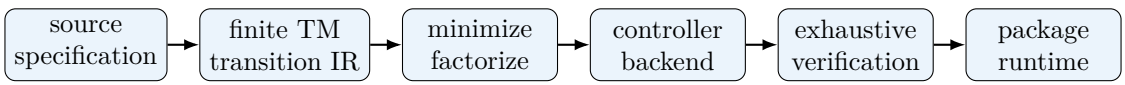

Figure 6: Compiler perspective: source → finite transition IR → exact or learned backend → verification → generic hard-tape runtime.

Possible front ends include:

- a text compiler that turns a finite document into a fixed- or variable-order context machine;
- a general TM compiler that accepts explicit states, alphabets, tape count, and transition rules;
- an algorithm compiler that lowers arithmetic, parsing, protocol control, or search procedures to explicit TM rules;

- a meta-machine compiler that combines several finite controllers behind a dispatcher.

Possible back ends include sparse constructive ReLU, dense constructive ReLU, exact quantized/integer ReLU, a compact gradient-trained ReLU, a verified learned ReLU, or a learned ReLU plus an exact exception table.

The generic runtime should not need to understand text n-grams, arithmetic, or proofs. Its job is only to manage the discrete memory mechanics and repeatedly execute

$$\texttt{READ} \rightarrow \texttt{controller} \rightarrow \texttt{WRITE} \rightarrow \texttt{MOVE}. \tag{31}$$

This also makes browser or mobile deployment conceptually simple. A compiled controller can be serialized as a compact binary or JSON-like artifact and evaluated by a generic JavaScript runtime. Sparse or quantized weights, Web Workers, and time-sliced execution can preserve responsiveness. The caveat is finite precision: exact mathematical weights should not be assumed exact after serialization. The deployed representation itself should be exhaustively checked whenever the local domain is enumerable.

11 Relationship to existing literature

The components of this framework are well precedented. The claim is not that neural simulation of computation, external memory, program execution, or model compression is new. The narrower synthesis is the use of an explicit finite TM transition function as an intermediate representation with constructive, learned, verified, and patched neural back ends.

Table 10: Comparison with selected related work.

Approach	Similarity	Main difference from the ReLU-TM compiler view	Ref.
Siegelmann–Sontag neural computation	Neural networks can simulate Turing computation	Broad computational-power result for recurrent analog/sigmoidal networks; not a practical finite-transition compiler with a hard external tape and exhaustive local verification	[1]
Omlin–Giles DFA construction	Known automata can be encoded constructively in neural weights; stability of long execution matters	Targets deterministic finite automata in second-order recurrent sigmoid networks; conceptually the closest precedent for direct rule-to-weight construction	[2]
Neural Turing Machines	Neural controller coupled to external memory	Memory access is differentiable/attentional and trained end-to-end; the present approach keeps reads, writes, and head moves hard and may compile known rules exactly	[3]
Neural Programmer-Interpreters	Neural core executes programs and uses scratchpad-like environments	Emphasizes learned compositional program execution from supervised traces; the present framework uses an explicit finite transition IR and exact verification when possible	[4]
Looped Transformers as programmable computers	Fixed neural weights can implement program counters, branches, and memory operations	Organizes computation inside a looped Transformer/instruction representation rather than as a specialized finite local TM controller	[5]
Looped ReLU MLP programmable computers	Shows strong programmable-computation capability of ReLU MLPs	Aims at a universal programmable computer; the ReLU-TM compiler specializes a finite transition map and outsources long-lived data to hard tapes	[6]

Approach	Similarity	Main difference from the ReLU-TM compiler view	Ref.
Knowledge distillation	A larger teacher can be compressed into a smaller learned student	Here the teacher is a fully enumerable finite transition relation, enabling exhaustive certification or exact patching of residual rows	[7]
Deep Compression	Pruning, quantization, and coding reduce network storage	These methods can be compiler back-end tools, but exact symbolic equivalence or full transition checks can be added because the target domain is finite	[8]
Classical automata minimization	Losslessly removes behaviorally redundant states	Purely symbolic; useful as an exact preprocessing pass before neural code generation	[9]
Neural policy + tree search (e.g. AlphaNPI)	Neural scores can guide symbolic/compositional search	The proof-gym result is much smaller and keeps a deterministic kernel as the source of legality; it establishes repair by search, not a state-of-the-art search advantage	[10]

Two comparisons deserve emphasis. First, Omlin and Giles already demonstrate the core idea that an automaton can be constructed into neural weights, so direct rule-to-weight encoding is not a novelty claim. Second, Neural Turing Machines and Neural Programmer-Interpreters are closer in spirit to learned algorithmic systems with external memory, but their memory and learning formulations are different from a hard discrete transition table that can be enumerated and verified.

12 What has been learned, and what has not?

The experiments repeatedly exposed several notions that should not be conflated.

12.1 Representability

The constructive theorems show that suitable weights exist for every finite deterministic local transition table. This is settled without training.

12.2 Learnability

A representable table may still be hard for a chosen architecture and optimizer. The learned monolithic ReaderFWNN on the 596-rule modular controller is direct evidence: exact weights exist, yet the compact trained checkpoint remained near 98% and failed long execution. The plain ReLU baseline, by contrast, found 596/596 reliably across ten seeds.

12.3 Compression

A direct one-row-per-detector realization is exact but not necessarily compact. Training can be interpreted as searching for parameter sharing that compresses the table. Verification then answers whether compression remained lossless. Exception patching provides a hybrid when it did not.

12.4 Algorithm discovery

The modular multiplication experiments do not show that the neural network discovered modular arithmetic from raw (a, b, p, answer) examples. The algorithm and its local transition targets were designed explicitly. Likewise, the writable Alice repair algorithm was designed symbolically and then distilled into a neural controller. A harder program-induction experiment would weaken or remove local rule supervision.

12.5 Long-run execution

Once the exact local controller is known, external tapes turn it into a scalable computation. The long-memory capacity comes from the tape, not from a hidden vector that somehow stores thousands of operand bits.

12.6 Text semantics

The high-order text machines can memorize finite text positions. The writable Alice controller can copy a clean source. Neither is evidence that the network has learned literary semantics. The source-swap experiment is especially decisive: when the clean source changes, the copied result changes with it.

12.7 Search heuristics

The proof policy learned useful tactic preferences but did not compositionally generalize under greedy decoding to two unseen motifs. Search/backtracking recovered all proofs. Yet random DFS was slightly better in median node count on the tiny held-out set, so no neural search-ordering advantage has been established.

Most useful conceptual separation: representability $\neq$ learnability $\neq$ compression $\neq$ algorithm discovery $\neq$ long-run execution.

13 Limitations and non-claims

1. **No free compression.** An arbitrary finite table with little reusable structure may require a neural artifact comparable in size to the table.
2. **Compilation is not discovery.** If the desired transition program is unknown, direct synthesis does not invent it. Program synthesis, search, learning, or human design is required upstream.
3. **High average local accuracy is unsafe for long deterministic runs.** The writable Alice v2 failure and the monolithic modular ReaderFWNN both demonstrate this empirically.
4. **Finite precision matters.** Mathematical constructions over real arithmetic are not automatically exact in FP32, FP16, INT8, JavaScript numbers, or hardware accelerators. Verification should target the deployed representation.
5. **Reachability matters.** Exactness on all defined rows is sufficient but can be stronger than necessary. Exactness on every reachable row from the intended start family is enough for the induction theorem, but proving reachability may itself be difficult.
6. **Cached neural decisions are an execution optimization.** The large modular runs cache the deterministic decoded 596-row table after verifying it. This is semantically equivalent to repeated identical forward passes, but should not be described as tens of millions of neural evaluations.
7. **Multi-tape machines change efficiency, not computability.** They are an engineering convenience for readable local controllers.
8. **The finite arithmetic JSON fixtures are not unbounded arithmetic implementations.** They are finite-domain empirical test fixtures. The later five-tape modular multiplier is length-independent, but its algorithm was manually engineered.
9. **The text experiments do not establish semantic language modeling.** High-order contexts memorize; short-order shared states provide only a controlled test bed for studying shared statistics.
10. **The proof gym is deliberately tiny.** It shows repair by exact search, not superiority to modern theorem provers or neural-guided proof systems.
11. **The working term is not standard nomenclature.** “ReLU Turing Machine” is a convenient label for this synthesis, not a claim of a new foundational machine model.

14 Research agenda

The compiler view suggests a more diagnostic agenda than simply asking whether a neural network can imitate a Turing machine.

14.1 Compression after correctness

For a fixed transition table, measure the Pareto frontier between exact constructive size, exact learned size, and learned-plus-exception size. Begin from a verified exact baseline, then prune width, depth, precision, and connectivity while repeatedly enumerating the complete local domain.

14.2 Quantized exactness

Search for low-bit integer or fixed-point representations whose deployed arithmetic is itself exhaustively certified. The copy-vs-memory INT8 result suggests large storage savings are possible, but exactness should be tested across the full relevant transition domain rather than only sampled rollouts.

14.3 Program induction with weaker supervision

Move from complete transition-table supervision to partial traces, sparse traces plus symbolic search, and finally end-to-end input/output objectives. Measure exactly when a model stops merely compressing a known table and begins discovering missing transition structure.

14.4 Cross-program sharing

Compile many related and unrelated programs jointly under a fixed parameter budget. Measure whether the learned controller discovers reusable subroutines or whether interference dominates. The finite dispatcher theorem guarantees representability with enough capacity but says nothing about efficient sharing.

14.5 Harder writable-text tasks

Remove the trivial clean-source channel. Natural next tasks are masked reconstruction from surrounding context, insertion/deletion editing, random-access retrieval across multiple snippets, and state-sequence semantic tests after genuine navigation has been established.

14.6 Text generalization rather than memorization

Scale the n-gram experiment to many books with a fixed small network. Train across document-specific machines, then evaluate on held-out passages and entirely new books. Only if a limited-capacity shared controller beats a pure memorization table on new documents is there evidence for shared learned language structure.

14.7 Proof search where ordering matters

Increase proof depth and add many distractor hypotheses so that exhaustive search becomes expensive. Compare policy-guided DFS, random DFS, BFS, beam search, and perhaps learned value functions under equal node budgets. The current 78/78 result is a correctness baseline, not an efficiency result.

14.8 Browser and mobile runtimes

Package exact or verified controllers as static artifacts for a generic JavaScript runner. Explore sparse matrix layouts, WebAssembly, Web Workers, time-sliced execution, and verified quantized weights. Compilation may be expensive while execution remains deterministic and lightweight.

15 Conclusion

A ReLU Turing Machine, in the working sense used here, is a conventional hard discrete tape machine whose finite local transition controller is implemented by a ReLU network. The useful contribution of this viewpoint is not a new claim of neural Turing completeness. It is a practical decomposition.

When a finite program is known, it can be compiled exactly into ReLU weights. When smaller storage is desired, the same finite transition relation becomes a complete supervised compression dataset. Because the local domain is finite, the deployed controller can be exhaustively checked; if a compact model misses a few rows, those rows can be patched exactly. Once local correctness is established, a discrete external tape supplies persistent working memory and the induction theorem transfers local exactness to arbitrarily long finite executions of the same deterministic program.

The experimental path matters because it shows where the clean theory does and does not help. Predictive text learning remained statistical and unstable in free run. A near-perfect controller failed catastrophically when one mandatory transition was wrong. A full arithmetic controller that was difficult for one architecture became easy for a plain ReLU. A writable text controller that reconstructed Alice turned out, under proper controls, to have learned copying rather than the book. A neural proof policy that failed greedily became reliable when placed inside an exact symbolic search loop, but did not yet outperform random search ordering.

The resulting design philosophy is therefore conservative: keep symbolic memory and legality where exactness matters; use neural networks as programmable or learnable finite controllers where they are useful; verify finite domains whenever possible; and distinguish carefully between encoding a program, learning a program table, compressing it, discovering an algorithm, and executing it for a long time.

Table 1: Condensed research path. Negative results are included because they drove the architectural changes.

Stage	Question	Main result	What changed next
0	Word-level text reader	Strong state prediction, weak word prediction; below bigram top-1	Move to characters and a cleaner finite-state representation
1	Character Alice reader	99.40% teacher-forced next-state accuracy, but only 31.91% next-character top-1; repetitive free run	Separate state-transition learning from language prediction
2	Plain ReLU baseline	Next-character top-1 rises to 43.94% (width 32) and 46.35% (width 128), but the complete table remains incomplete	Ask whether finite transition tables can be represented exactly
3	Constructive theorem	Any finite deterministic transition table has an explicit exact ReLU realization	Treat the TM table as a compiler IR
4	Arithmetic fixtures	Exact hand-set ReLU on 1,781 rows; exact hand-set ReaderFWNN on 156 rows; learned shared FWNN still not exact	Study the exactness threshold on a long algorithm
5	Modular multiplication	Learned ReaderFWNN near 98% fails; plain ReLU learns 596/596 rules in 10/10 seeds and runs to 4000-bit cases	Make writable memory central rather than incidental
6	Writable Alice repair	99.962% local accuracy gives 0/500 rollouts; coverage-augmented model gives 700/700 exact rollouts	Test whether the network memorized Alice or learned a generic procedure
7	Copy-vs-memory control	2000/2000 random OOD strings copied under the trained corruption family; source swap proves source-driven copying	Seek tasks where source structure, not a trivial clean channel, is necessary
8	Text-machine theory	n-gram order gives a controlled continuum from shared statistics to deterministic memorization	Formulate multi-document and capacity experiments
9	Symbolic proof gym	Greedy neural policy fails unseen motifs; exact DFS/backtracking repairs the policy to 78/78	Generalize the hybrid pattern: learned heuristic + exact symbolic kernel

Table 2: A practical exactness-size spectrum.

Mode	Goal	Guarantee	Typical cost
Construct exact	Synthesize weights directly from δ	Exact by construction	Potentially large
Optimize exact	Minimize, factorize, sparsify, quantize without changing behavior	Exact after equivalence or transition checks	Smaller when structure exists
Learn + verify	Train a compact controller and enumerate every row	Exact only if all checks pass	Can be much smaller
Learn + patch	Train compactly and store residual exception rows	Exact combined system	Small network + small table
Approximate	Enforce a hard model-size budget	No exactness guarantee	Potentially smallest artifact

Table 3: Selected statistics from the two-text context experiment. H is conditional next-character entropy in bits. "Conflicts" counts shared contexts whose modal successor differs across the two texts.

n	Hesse top-1	Hesse det.	Hesse H	Goethe top-1	Goethe det.	Goethe H	Conflicts
2	64.8%	51.2%	1.067	77.9%	73.2%	0.565	29
3	80.6%	78.4%	0.509	90.0%	89.2%	0.220	23
4	91.4%	90.9%	0.198	95.7%	95.2%	0.086	15
5	94.7%	94.7%	0.124	97.9%	97.7%	0.043	5
6	97.0%	96.7%	0.065	97.9%	97.8%	0.043	3
8	99.5%	99.5%	0.009	100.0%	100.0%	0.000	2
10	99.8%	99.7%	0.005	100.0%	100.0%	0.000	1
16	99.8%	99.8%	0.005	100.0%	100.0%	0.000	1
24	99.8%	99.8%	0.005	100.0%	100.0%	0.000	1
32	100.0%	100.0%	0.000	100.0%	100.0%	0.000	1

Table 4: Exact continuation rate from random genuine seeds.

n	Hesse exact	Hesse mean prefix	Goethe exact	Goethe mean prefix
2	0.0%	2.1/80	0.0%	4.2/35
3	0.0%	5.4/80	1.5%	8.8/35
4	0.0%	10.6/80	57.5%	26.1/35
5	9.0%	26.3/80	73.5%	28.6/35
6	22.0%	41.8/80	72.5%	27.4/35
8	72.5%	66.2/80	100.0%	35.0/35
10	92.0%	74.6/80	100.0%	35.0/35
16	90.5%	73.7/80	100.0%	35.0/35
24	93.5%	75.7/80	100.0%	35.0/35
32	100.0%	80.0/80	100.0%	35.0/35

Table 5: Held-out Alice comparison. "Complete table" evaluates all 128×49 state-symbol cells.

Model	State acc.	Char top-1	Char top-5	PPL	Complete table
Growing FWNN, support 24	99.40%	31.91%	66.55%	11.42	69.39%
ReLU-32, matched exposure	99.83%	39.65%	72.02%	8.89	75.41%
ReLU-32, 2500 updates	99.96%	43.94%	76.83%	7.16	77.49%
ReLU-128, 2500 updates	99.95%	46.35%	80.86%	6.03	77.73%

Table 6: Selected large runs using the constructively exact ReaderFWNN controller. Step counts are for the particular random cases in this run.

Operand bits	Modulus bits	TM steps	Correct
1,024	521	6,537,382	yes
2,048	1,021	25,122,838	yes
3,333	1,729	69,026,765	yes
3,991	2,048	98,279,693	yes
4,000	2,048	98,677,460	yes

Table 7: Ten independent plain-ReLU training runs. Every run reached the exact 596-row table.

Seed	1	2	3	4	5	101	202	303	404	505
Steps	450	450	425	475	525	550	400	375	475	500

Table 8: Fresh random modular-multiplication runs using the learned exact plain-ReLU transition table. These are different random cases from table 6, hence the step counts differ.

Operand bits	Modulus bits	TM transitions	Correct
117	53	77,169	yes
309	248	912,245	yes
1,523	689	12,775,955	yes
3,066	1,857	68,256,966	yes
3,333	1,729	69,193,099	yes
3,991	2,048	98,378,073	yes
4,000	2,048	98,968,814	yes

Table 9: Compression check from the copy-vs-memory addendum. This is not a Kolmogorov-complexity estimate.

Object	Raw bytes	gzip bytes	gzip/raw
Project Gutenberg Alice file	174,311	60,934	0.350
Normalized Alice text	143,222	49,134	0.343
PyTorch state dict	148,645	134,299	0.903
Raw FP32 tensor bytes	143,092	132,255	0.924
FP16 tensor bytes	71,546	65,746	0.919
INT8 tensors + per-tensor scales	35,829	30,727	0.858

A Consolidated source notes and artifact packages

This draft consolidates the following working notes and reproducibility artifacts produced in August 2026. They are listed here to make the provenance of the internal experimental numbers explicit.

Working notes

- `What_Is_a_ReLU_Turing_Machine.pdf` — exact compilation, compression, verification, compiler view, applications, limitations, and literature positioning.
- `ReLU_Turingmaschinen_zum_Lernen_von_Texten.pdf` — n-gram text machines, exact finite-text memorization, multi-text conditions, Goethe/Hesse experiments.
- `Neural_Controllers_with_Writable_Tapes_Alice_SAIR.pdf` — Alice predictive reader, plain ReLU comparison, 596-rule modular multiplication, large rollouts, and exactness discussion.
- `Alice_on_a_Writable_Tape.pdf` — random-head repair, bidirectional movement and writing, v2/v3/v5 training progression, end-to-end rollout results.
- `Alice_Writable_Tape_Copy_vs_Memory_Addendum.pdf` — OOD copying, source-swap control, and compression check.
- `tm_fwnn_theorem.md` — explicit proofs of exact ReLU transition-table simulation, whole-run induction, ReaderFWNN constructive existence, and finite multi-machine dispatch.

Artifact packages

- `alice_tm_fwnn_pilot.zip` — original word- and character-level Alice ReaderFWNN code, induced states, metrics, growth traces, and checkpoint.
- `tm_fwnn_proof_and_tests.zip` — JSON TMs, exact ReLU compiler, manual ReaderFWNN construction, arithmetic fixtures, dispatcher, and joint FWNN experiment.
- `sair_fwnn_modmul_full_controller.zip` — four primitive arithmetic TMs, complete five-tape modular multiplier, learned and constructive ReaderFWNN controllers, and large-case results.
- `sair_plain_relu_tm.zip` — 59–64–64–64 plain ReLU controller, ten-seed exact-learning runs, exhaustive table verification, and large random rollouts.
- `mini_symbolic_proof_gym_v1.zip` — symbolic proof kernel, BFS oracle traces, ReLU tactic policy, v0/v1 representational change, greedy/DFS/BFS comparisons, and checkpoints.

B Selected reproducibility facts

Artifact	Reproducibility fact
Alice character pilot	143,127 normalized characters; 128 induced states; 49-character alphabet; final support 24.
Exact arithmetic compilation	14 main machines; 116 main cases; 873 disjoint states; 1,781 defined rows; exact hand-set ReLU at 100% state/action.
Manual ReaderFWNN check	Base-2 addition + binary multiplication; 112 states; 156 transitions; 155 supports; $R = 77$; latent dimension 156; exact state/action.
Full modular controller	Five tapes; 39 states; 596 defined rows.
Learned monolithic ReaderFWNN	40 supports; 534 observed short-curriculum rows; 518 exact; about 97.6% state and 97.8% action; first long case failed.
Manual full ReaderFWNN	595 supports; $R = 297$; exact on all 596 rows; fresh 4000/2048-bit case correct.
Plain ReLU full controller	16,970 parameters; all 10 seeds reached 596/596; zero mismatches in the decoded deployed table.
Writable Alice v5	35,773 parameters; 15,872 distinct held-out local inputs; 100% all four local heads; 500/500 standard + 200/200 stress exact.

Copy-vs-memory addendum	2000/2000 random OOD copies under the demonstrated corruption family; 63.5% under adversarial arbitrary work symbols.
Proof gym v1	332 training theorems; 84 seen-family holdout; 78 wholly held-out-family tests; greedy 26/78 on new families; guided DFS, random DFS, and BFS each 78/78.

C External references

References

[1] H. T. Siegelmann and E. D. Sontag. On the computational power of neural nets. *Journal of Computer and System Sciences*, 50(1):132–150, 1995. doi:10.1006/jcss.1995.1013.

[2] C. W. Omlin and C. L. Giles. Constructing deterministic finite-state automata in recurrent neural networks. *Journal of the ACM*, 43(6):937–972, 1996. doi:10.1145/235809.235811.

[3] A. Graves, G. Wayne, and I. Danihelka. Neural Turing Machines. arXiv:1410.5401, 2014.

[4] S. Reed and N. de Freitas. Neural Programmer-Interpreters. arXiv:1511.06279, 2015; presented at ICLR 2016.

[5] A. Giannou, S. Rajput, J.-Y. Sohn, K. Lee, J. D. Lee, and D. Papailiopoulos. Looped Transformers as Programmable Computers. In *Proceedings of ICML 2023*, PMLR 202:11398–11442, 2023.

[6] Y. Liang, Z. Sha, Z. Shi, Z. Song, and Y. Zhou. Looped ReLU MLPs May Be All You Need as Practical Programmable Computers. In *Proceedings of AISTATS 2025*, PMLR 258:2647–2655, 2025.

[7] G. Hinton, O. Vinyals, and J. Dean. Distilling the Knowledge in a Neural Network. arXiv:1503.02531, 2015.

[8] S. Han, H. Mao, and W. J. Dally. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding. ICLR 2016; arXiv:1510.00149.

[9] J. E. Hopcroft. An $n \log n$ algorithm for minimizing states in a finite automaton. In *Theory of Machines and Computations*, pp. 189–196, 1971. doi:10.1016/B978-0-12-417750-5.50022-1.

[10] T. Pierrot, G. Ligner, S. Reed, O. Sigaud, N. Perrin, A. Laterre, D. Kas, K. Beguir, and N. de Freitas. Learning Compositional Neural Programs with Recursive Tree Search and Planning. arXiv:1905.12941, 2019.